

Milestone 2.3: Clustering

INFO 521 · Machine Learning Foundations · project milestone brief ·
From Inference to Discovery · Gate B (Unsupervised) · 1 of 2

How to read this

This is the assignment brief for one project milestone. The Satisfactory criteria below are authoritative: every one must be met, and it is all-or-nothing rather than partial credit. A Not-Yet returns with targeted feedback and one revise cycle. Do the work in the notebook distributed through Classroom 50; this PDF is the brief, not the workspace.

Course-schedule placement. Part 2 of the project: begins at the midpoint, due Week 7.5. This milestone covers M6 and is due Week 7. Gated by Checkpoint quiz 2.3 (k-means objective / PCA, Wk 6); Gate B.

Drop the labels

Gate A *predicted* a label you were handed. Gate B asks a different question: with **no label at all**, is there structure in these patients to be *discovered*? You will run **k-means** on the same six-feature space, never showing it the hypertension label, and only *afterward* bring the label back to ask the payoff question: do the subgroups the algorithm found on its own line up with the clinical strata? This is the supervised → unsupervised turn of the whole project.

Ground rules. Implement k-means and every selection metric from scratch in NumPy/SciPy. No scikit-learn, no KMeans, no `silhouette_score`. The `info521` library gives you data, plotting, and self-checks only.

What's a feature here. The clustering inputs are the **six non-BP columns** `ds["X"]` (age, bmi, waist, chol, hdl, hba1c). The hypertension label is **held out of the clustering entirely**, it appears only in §4, the reveal. Unlike Gate A there is no train/test split: we are *describing structure in the whole cohort*, so standardize the full feature matrix.

What “Satisfactory” means for 2.3

- ☐ You implement **k-means from scratch** (the **assignment** step and the **update** step) on the **standardized** six-feature space, with an explicit **within-cluster sum of squares (WCSS)** objective.
- ☐ You address **local minima** with **multiple random restarts**, keeping the solution with the lowest WCSS.
- ☐ You perform **model selection for K** (elbow and/or silhouette), with the selection metric computed **from scratch**.

- ☐ You set up the **reveal**: cross-tabulate the discovered clusters against the **held-out hypertension label**, report the between-cluster **relative risk** (enrichment) as the headline alignment metric, and discuss whether unsupervised subgroups recover clinical strata.
 - ☐ **Play artifact** (below) complete and interpreted.
 - ☐ **Checkpoint quiz 2.3 (k-means objective & choosing K)** passed.
-

Setup

```
import numpy as np
from info521 import data, plotting, checks

plotting.use_course_style()
ds = data.load_clinical()

X_raw = ds["X"] # (n, 6): age, bmi, waist, chol, hdl, hba1c
feature_names = ds["features"]
labels, rule = data.hypertension(ds) # SET ASIDE, used only in §4, never to cluster

print(f"{X_raw.shape[0]} patients · {X_raw.shape[1]} features = {feature_names}")
print(f"(held-out label for the §4 reveal: {rule})")
```

0 · Standardize the feature space

k-means uses Euclidean distance, so features on larger scales (e.g. chol in the hundreds) would dominate hba1c (units of ~1). Put every feature on a common footing first.

```
def standardize(X):
    # TODO: return the z-scored matrix (subtract each column's mean, divide by sd).
    raise NotImplementedError

# TODO: Xz = standardize(X_raw)
```

1 · k-means from scratch

Pick K centroids, then alternate to convergence:

- **Assignment.** Assign each point to its nearest centroid (squared Euclidean distance): $z_n = \arg \min_k \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$.
- **Update.** Move each centroid to the mean of its assigned points: $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{n \in C_k} \mathbf{x}_n$.

The objective both steps decrease is the **within-cluster sum of squares**

$$J = \sum_{k=1}^K \sum_{n \in C_k} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2.$$

```
def kmeans(X, K, n_iter=100, seed=521):
    # TODO: initialize K centroids (e.g. K random distinct rows), then loop
    # assignment + update until labels stop changing or n_iter is hit.
    # Return (labels, centroids, wcss). Decide how to handle an empty cluster.
    raise NotImplementedError
```

```
def wcss(X, labels, centroids):
    # TODO: total within-cluster sum of squared distances (the J above).
    raise NotImplementedError
```

2 · Local minima: random restarts

k-means converges to a *local* optimum that depends on the initialization. Run it several times from different random starts and **keep the lowest-WCSS** solution.

```
def kmeans_best_of(X, K, n_restarts=10, seed=521):
    # TODO: run kmeans n_restarts times with different seeds; return the result
    # with the smallest WCSS. Also collect every restart's WCSS so you can show
    # how much the objective varies across starts (the local-minima story).
    raise NotImplementedError
```

3 · Choosing K (from scratch)

K is yours to choose, and minimizing WCSS alone is a trap (more on that in Checkpoint 2.3). Use at least one principled criterion, implemented from scratch:

- **Elbow.** Plot best WCSS versus K and look for the diminishing-returns bend.
- **Silhouette.** For point n let a_n be its mean distance to its own cluster and b_n the mean distance to the nearest *other* cluster; then $s_n = (b_n - a_n) / \max(a_n, b_n)$. Average s_n over all points and pick the K that maximizes it.

```
def silhouette(X, labels):
    # TODO: compute the mean silhouette score from scratch (no sklearn).
    raise NotImplementedError

# TODO: sweep K (e.g. 2..8); plot the elbow (WCSS vs K) and/or mean silhouette vs
# K; state which K you choose and why.
```

4 · The reveal: clusters vs the held-out label

Now, and *only* now, bring back the hypertension label you set aside. Cross-tabulate your chosen clustering against it, then summarize the alignment with the **right** metric.

Headline the enrichment, not purity. Cluster *purity* (how often each cluster's majority label is correct) is dominated by the base rate: with ~39% prevalence, a clustering that learned *nothing* still scores ~61% purity, so purity hides real signal. The honest headline is the **enrichment / relative risk (RR)**, the hypertension rate in the most-hypertensive cluster divided by that in the least-hypertensive cluster. An RR meaningfully above 1 means the clusters *do* carry hypertension signal, even when purity barely moves off the base rate.

```
def crosstab(cluster_labels, binary_label):
    # TODO: build the K x 2 contingency table (rows = clusters, cols = {0,1}).
    # Report, per cluster, the fraction that is hypertensive.
    raise NotImplementedError

# TODO: cross-tab your best K-cluster solution against `labels`. Report each
```

```
# cluster's hypertension fraction AND the between-cluster RELATIVE RISK
# (max fraction / min fraction) as your headline alignment metric.
```

Write 200–300 words, leading with the relative risk: did the unsupervised subgroups recover the clinical strata? If a cluster is enriched for hypertension, *which features* (from §0's standardized space) drive it, and does that match the correlations you saw back in Part 1's data? Be honest about the gap: a real $RR > 1$ alongside near-base-rate purity means k-means found *geometric* structure (body composition, age) that is **correlated with** but does not **equal** the *clinical* blood-pressure stratum.

Play artifact: K , restarts, and the cross-tab

1. **Objective stability.** For a fixed K , plot the spread of WCSS across many restarts, how often does k-means land in a worse local minimum?
2. **Vary K .** Show the elbow/silhouette curves and how the cluster-vs-label cross-tab changes as K grows (do finer clusters sharpen the hypertension separation, or just fragment it?).

Write 150–250 words connecting the two: how restart stability and the choice of K together determine how much you should trust the discovered structure.

```
# TODO: restart-stability plot + K sweep + cross-tab evolution + interpretation.
```

Leakage demonstration (optional, ungraded)

! Warning

Not part of Satisfactory. This is a deliberately *wrong* analysis, run once to see what label leakage looks like. Do it only after the real analysis above; it earns no credit and must not replace §4.

The hypertension label is defined from blood pressure, and your features deliberately **exclude** sbp/dbp to keep the reveal honest. Watch what happens if you cheat and add the BP columns back into the clustering inputs, then recompute the **same** between-cluster relative risk from §4:

```
# TODO (optional): build an 8-column matrix = the six features PLUS sbp and dbp,
#   X_leak = np.column_stack([X_raw, ds["y"], ds["dbp"]])
# standardize it, re-run kmeans_best_of, and recompute the SAME relative-risk
# metric. Compare RR_leak to the honest RR from §4.
```

You should see the RR **jump sharply**: the clusters now split almost along the 130/80 thresholds that *define* the label, so they “recover” hypertension almost perfectly. That is not insight, it is **label leakage**. The clusters recovered the label only because the label's own ingredients were handed to them. This is exactly what leakage looks like, and exactly why §0 here and every supervised notebook in Gate A exclude the BP columns. The honest RR from §4 (modest, driven by the metabolic/age channel) is the real result.

i Gate

This milestone is not Satisfactory until **Checkpoint quiz 2.3 (state the k-means objective, explain why minimizing it over K trivially fails, and give one principled K -selection method)** is passed. The checkpoint is an open-book D2L quiz, due the week before this milestone; the auto-graded portion resolves on submission and written responses allow one revision. Checkpoint quiz 2.3 gates Gate B as a whole.

Looking ahead

In **2.4** you compress these six features to two **principal components**, then re-view *these very clusters* in that 2-D space (the “reduce, then look” payoff) and read which features drive the axes along which the cohort actually varies.